Gateway 프록시 예제

개요

이 예제는 IPFS Gateway를 사용하여 HTTP 프록시 서버를 구현하는 방법을 보여줍니다. 두 가지 주요 백엔드 방식을 다룹니다:

1. **블록 기반 프록시**: 원격 게이트웨이에서 개별 블록을 가져와 검증
2. **CAR 기반 프록시**: 원격 게이트웨이에서 CAR 파일을 가져와 검증

이러한 프록시는 신뢰할 수 없는 원격 게이트웨이로부터 데이터를 안전하게 가져오면서도 완전한 웹 게이트웨이 기능을 제공합니다.

주요 개념

Gateway 프록시 아키텍처

1. **웹 브라우저**
    - HTTP 요청을 보냄
2. **Gateway 프록시**
    - 웹 브라우저로부터 HTTP 요청을 수신
    - 원격 게이트웨이에 블록/CAR 요청을 전달
    - 원격 게이트웨이로부터 받은 원시 데이터를 **로컬 검증 및 처리** 모듈을 통해 검증/가공
3. **원격 게이트웨이**
    - Gateway 프록시로부터 받은 블록/CAR 요청을 처리
    - 원시 데이터를 반환

4. **웹 브라우저**
    - Gateway 프록시에서 최종적으로 검증/가공된 HTML/파일 응답을 수신

즉 순서를 간단히 표현하면:

`웹 브라우저 (HTTP 요청) → Gateway 프록시 (블록/CAR 요청) → 원격 게이트웨이 (원시 데이터) → Gateway 프록시 (로컬 검증/처리 후 응답) → 웹 브라우저 (HTML/파일)`

핵심 구성 요소

1. **Gateway Handler**: HTTP 요청을 처리하고 응답을 생성
2. **Backend**: 원격 데이터 소스와의 통신 담당
3. **Verification**: 가져온 데이터의 무결성 검증
4. **Routing**: IPNS 레코드 및 DNSLink 해석

블록 기반 프록시 구현

1. 기본 설정 및 의존성

```go
package main


import (
"context"
"flag"
"fmt"
"log"
"net/http"
```

```go
"strconv"
"time"

"github.com/ipfs/boxo/gateway"
"github.com/ipfs/boxo/gateway/assets"
"github.com/prometheus/client_golang/prometheus"
"github.com/prometheus/client_golang/prometheus/promhttp"
"go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
)

const (
defaultPort = 8040
defaultTimeout = 30 * time.Second
)
```

2. 블록 기반 프록시 서버

```go
// BlockProxyServer는 블록 기반 프록시 서버를 구현합니다
type BlockProxyServer struct {
backend gateway.IPFSBackend
handler http.Handler
port  int
}
```

```go
// NewBlockProxyServer는 새로운 블록 기반 프록시 서버를 생성합니다
func NewBlockProxyServer(gatewayURLs []string, port int) (*BlockProxyServer, e
if len(gatewayURLs) == 0 {
return nil, fmt.Errorf("최소 하나의 게이트웨이 URL이 필요합니다")
}

// 원격 블록 백엔드 생성
// 이 백엔드는 application/vnd.ipld.raw 형식으로 블록을 요청합니다
backend, err := gateway.NewRemoteBlocksBackend(gatewayURLs, nil)
if err != nil {
return nil, fmt.Errorf("원격 블록 백엔드 생성 실패: %w", err)
}

// Gateway 핸들러 생성
handler := createGatewayHandler(backend)

return &BlockProxyServer{
backend: backend,
handler: handler,
port: port,
}, nil
}
```

```go
// Start는 프록시 서버를 시작합니다
func (bps *BlockProxyServer) Start(ctx context.Context) error {
server := &http.Server{
Addr: fmt.Sprintf(":%d", bps.port),
Handler: bps.handler,
ReadTimeout: defaultTimeout,
WriteTimeout: defaultTimeout,
}

log.Printf("블록 기반 프록시 서버 시작: http://localhost:%d", bps.port)
log.Printf("이미지 테스트: http://localhost:%d/ipfs/bafybeigdyrzt5sfp7udm7hu76uh7y
log.Printf("위키피디아 테스트: http://localhost:%d/ipfs/bafybeiaysi4s6lnjev27ln5icv
log.Printf("메트릭: http://127.0.0.1:%d/debug/metrics/prometheus", bps.port)

// 컨텍스트 취소 시 서버 종료
go func() {
<-ctx.Done()
log.Println("서버 종료 중...")
shutdownCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
if err := server.Shutdown(shutdownCtx); err != nil {
log.Printf("서버 종료 오류: %v", err)
```

```
}
}()

if err := server.ListenAndServe(); err != http.ErrServerClosed {
return fmt.Errorf("서버 시작 실패: %w", err)
}

return nil
}
```

3. CAR 기반 프록시 서버

```
// CARProxyServer는 CAR 기반 프록시 서버를 구현합니다
type CARProxyServer struct {
backend gateway.IPFSBackend
handler http.Handler
port int
}

// NewCARProxyServer는 새로운 CAR 기반 프록시 서버를 생성합니다
func NewCARProxyServer(gatewayURLs []string, port int) (*CARProxyServer, error
if len(gatewayURLs) == 0 {
```

```go
return nil, fmt.Errorf("최소 하나의 게이트웨이 URL이 필요합니다")
}

// 원격 CAR 백엔드 생성
// 이 백엔드는 application/vnd.ipld.car 형식으로 데이터를 요청합니다
// IPIP-0402: Partial CAR Support를 구현하는 게이트웨이가 필요합니다
backend, err := gateway.NewRemoteCarBackend(gatewayURLs, nil)
if err != nil {
return nil, fmt.Errorf("원격 CAR 백엔드 생성 실패: %w", err)
}

// Gateway 핸들러 생성
handler := createGatewayHandler(backend)

return &CARProxyServer{
backend: backend,
handler: handler,
port: port,
}, nil
}

// Start는 CAR 프록시 서버를 시작합니다
func (cps *CARProxyServer) Start(ctx context.Context) error {
```

```
server := &http.Server{
Addr: fmt.Sprintf(":%d", cps.port),
Handler: cps.handler,
ReadTimeout: defaultTimeout,
WriteTimeout: defaultTimeout,
}


log.Printf("CAR 기반 프록시 서버 시작: http://localhost:%d", cps.port)
log.Printf("이미지 테스트: http://localhost:%d/ipfs/bafybeigdyrzt5sfp7udm7hu76uh7y
log.Printf("위키피디아 테스트: http://localhost:%d/ipfs/bafybeiaysi4s6lnjev27ln5icv
log.Printf("메트릭: http://127.0.0.1:%d/debug/metrics/prometheus", cps.port)


// 컨텍스트 취소 시 서버 종료
go func() {
<-ctx.Done()
log.Println("서버 종료 중...")
shutdownCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
if err := server.Shutdown(shutdownCtx); err != nil {
log.Printf("서버 종료 오류: %v", err)
}
}()
```

```
if err := server.ListenAndServe(); err != http.ErrServerClosed {
return fmt.Errorf("서버 시작 실패: %w", err)
}

return nil
}
```

4. 공통 Gateway 핸들러 생성

```
// createGatewayHandler는 공통 게이트웨이 핸들러를 생성합니다
func createGatewayHandler(gwAPI gateway.IPFSBackend) http.Handler {
// Gateway 설정
conf := gateway.Config{
// DNSLink 지원 활성화
// DNSLink가 CAR의 CID를 가리키도록 설정하면 로드할 수 있습니다
NoDNSLink: false,

// 신뢰 모드 활성화 - 모든 유형의 요청을 허용
// 기본적으로는 Trustless Gateway 요청만 작동합니다
// https://specs.ipfs.tech/http-gateways/trustless-gateway/
DeserializedResponses: true,

// 다양한 호스트명에서 사용할 공개 게이트웨이 초기화
```

```
// 이 단계는 선택사항이지만, 다른 호스트명에서 여러 공개 게이트웨이를
// 실행하고 다른 설정을 원하는 경우 필요합니다
PublicGateways: map[string]*gateway.PublicGateway{
// Host: CID.ipfs.example.net 및 ID.ipns.example.net 공개 요청 지원
"example.net": {
Paths: []string{"/ipfs", "/ipns"},
NoDNSLink: false,
UseSubdomains: true,
// 이 서브도메인 게이트웨이는 테스트용이므로 비신뢰 요청을 허용합니다
DeserializedResponses: true,
},
// 로컬 요청 지원
"localhost": {
Paths: []string{"/ipfs", "/ipns"},
NoDNSLink: false,
UseSubdomains: true,
// localhost는 신뢰할 수 있는 것으로 간주되므로
// 인터넷에 노출되지 않는 한 역직렬화된 응답을 허용합니다
DeserializedResponses: true,
},
},

// 'Boxo' 라이브러리로 링크되는 예제 메뉴 항목 추가
```

```
Menu: []assets.MenuItem{
{
URL: "https://github.com/ipfs/boxo",
Title: "Boxo",
},
},
}

// 게이트웨이 경로를 제공하는 mux 생성
// 이는 엄격히 필요하지 않으며 gwHandler를 직접 사용할 수 있습니다
// 하지만 다음 단계에서 prometheus 메트릭도 추가하려고 하므로 mux가 필요합니다
gwHandler := gateway.NewHandler(conf, gwAPI)
mux := http.NewServeMux()
mux.Handle("/ipfs/", gwHandler)
mux.Handle("/ipns/", gwHandler)

// 게이트웨이와 함께 prometheus 메트릭 제공
// 이 단계는 선택사항이며 메트릭에 액세스해야 하거나 원하는 경우에만 필요합니다
// 다른 경로나 포트에서 메트릭을 노출하기로 결정할 수도 있습니다
mux.Handle("/debug/metrics/prometheus", promhttp.HandlerFor(prometheus.DefaultGa

// 그런 다음 호스트명 핸들러로 mux를 래핑합니다
// 메트릭은 이전에 정의된 publicGateways에서 사용할 수 없습니다
```

```
// 127.0.0.1을 통해 메트릭에 액세스할 수 있지만 localhost나 example.net은 불가능합니다
// 이러한 게이트웨이에서 메트릭을 노출하려면 Paths 변수에 "/debug" 경로를 추가해야 합니다
var  handler http.Handler
handler = gateway.NewHostnameHandler(conf, gwAPI, mux)

// 그런 다음 withConnect 미들웨어로 래핑합니다
// http.ServeMux는 기본적으로 CONNECT를 지원하지 않으므로 이것이 필요합니다
handler = withConnect(handler)

// 모든 요청에 정의한 헤더를 적용하는 헤더 미들웨어와
// 기본 CORS 구성을 추가합니다
handler = gateway.NewHeaders(nil).ApplyCors().Wrap(handler)

// 마지막으로 otelhttp 핸들러로 래핑합니다
// 이를 통해 추적 시스템이 작동하고 추적 헤더가 올바르게 전파됩니다
// 이 단계는 선택사항이며 추적을 사용하려는 경우에만 필요합니다
// 이것이 효과를 가지려면 OTel이 올바르게 설정되어야 합니다
handler = otelhttp.NewHandler(handler, "Gateway.Request")

return  handler
}

// withConnect는 HTTP CONNECT 메서드에 대한 지원을 추가하는 미들웨어를 제공합니다
```

```go
// 구현자가 http.ServeMux 또는 CONNECT 메서드를 지원하지 않는 다른 구조를 사용하는 경우
// 최상위 핸들러에 적용되어야 합니다
func withConnect(next http.Handler) http.Handler {
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
if r.Method == http.MethodConnect {
w.WriteHeader(http.StatusOK)
return
}

next.ServeHTTP(w, r)
})
}
```

5. 메인 함수 및 실행 로직

```go
func main() {
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

// 명령줄 플래그 정의
var (
gatewayURL = flag.String("g", "", "프록시할 게이트웨이 URL (필수)")
```

```
port = flag.Int("p", defaultPort, "서버 실행 포트")

proxyType = flag.String("type", "blocks", "프록시 타입: 'blocks' 또는 'car'")

help = flag.Bool("h", false, "도움말 표시")

)

flag.Parse()


if *help {

printUsage()

return

}


if *gatewayURL == "" {

log.Fatal("게이트웨이 URL이 필요합니다. -g 플래그를 사용하세요.")

}


gatewayURLs := []string{*gatewayURL}


// 프록시 타입에 따라 서버 생성 및 시작

switch *proxyType {

case  "blocks":

server, err := NewBlockProxyServer(gatewayURLs, *port)

if  err != nil {

log.Fatalf("블록 프록시 서버 생성 실패: %v", err)
```

```
}
if err := server.Start(ctx); err != nil {
log.Fatalf("블록 프록시 서버 시작 실패: %v", err)
}

case "car":
server, err := NewCARProxyServer(gatewayURLs, *port)
if err != nil {
log.Fatalf("CAR 프록시 서버 생성 실패: %v", err)
}
if err := server.Start(ctx); err != nil {
log.Fatalf("CAR 프록시 서버 시작 실패: %v", err)
}

default:
log.Fatalf("지원하지 않는 프록시 타입: %s. 'blocks' 또는 'car'를 사용하세요.", *proxy
}
}

func printUsage() {
fmt.Println("IPFS Gateway 프록시 서버")
fmt.Println()
fmt.Println("사용법:")
```

```
    fmt.Println(" gateway-proxy -g <게이트웨이_URL> [옵션]")

    fmt.Println()

    fmt.Println("옵션:")

    fmt.Println(" -g string")

    fmt.Println(" 프록시할 게이트웨이 URL (필수)")

    fmt.Println(" -p int")

    fmt.Println(" 서버 실행 포트 (기본값: 8040)")

    fmt.Println(" -type string")

    fmt.Println(" 프록시 타입: 'blocks' 또는 'car' (기본값: blocks)")

    fmt.Println(" -h 도움말 표시")

    fmt.Println()

    fmt.Println("예제:")

    fmt.Println(" # 블록 기반 프록시")

    fmt.Println(" gateway-proxy -g https://trustless-gateway.link -type blocks")

    fmt.Println()

    fmt.Println(" # CAR 기반 프록시")

    fmt.Println(" gateway-proxy -g https://trustless-gateway.link -type car -p 8080"

    }
```

고급 기능 구현

1. 다중 백엔드 지원

```
// MultiBackendProxy는 여러 백엔드를 지원하는 프록시입니다
type MultiBackendProxy struct {
backends []gateway.IPFSBackend
handler http.Handler
port  int
}

// NewMultiBackendProxy는 여러 백엔드를 사용하는 프록시를 생성합니다
func  NewMultiBackendProxy(gatewayURLs []string, port  int, useCAR  bool) (*Mult
if  len(gatewayURLs) == 0 {
return  nil, fmt.Errorf("최소 하나의 게이트웨이 URL이 필요합니다")
}

var  backends []gateway.IPFSBackend

// 각 URL에 대해 백엔드 생성
for  _, url := range  gatewayURLs {
var  backend gateway.IPFSBackend
var  err  error

if  useCAR {
backend, err = gateway.NewRemoteCarBackend([]string{url}, nil)
```

```
} else {
backend, err = gateway.NewRemoteBlocksBackend([]string{url}, nil)
}

if err != nil {
log.Printf("백엔드 생성 실패 (%s): %v", url, err)
continue
}

backends = append(backends, backend)
log.Printf("백엔드 추가됨: %s", url)
}

if len(backends) == 0 {
return nil, fmt.Errorf("사용 가능한 백엔드가 없습니다")
}

// 첫 번째 백엔드를 기본으로 사용 (실제로는 로드 밸런싱 로직 필요)
handler := createGatewayHandler(backends[0])

return &MultiBackendProxy{
backends: backends,
handler: handler,
```

```
port: port,

}, nil

}
```

2. 캐싱 지원

```
// CachingProxy는 캐싱 기능을 포함한 프록시입니다

type CachingProxy struct {

backend gateway.IPFSBackend

cache  map[string][]byte  // 간단한 메모리 캐시

handler http.Handler

port  int

mu sync.RWMutex

}


// NewCachingProxy는 캐싱 기능을 포함한 프록시를 생성합니다

func  NewCachingProxy(gatewayURLs []string, port  int) (*CachingProxy, error) {

backend, err := gateway.NewRemoteBlocksBackend(gatewayURLs, nil)

if  err != nil {

return  nil, fmt.Errorf("백엔드 생성 실패: %w", err)

}


proxy := &CachingProxy{
```

```
backend: backend,
cache: make(map[string][]byte),
port: port,
}

// 캐싱 미들웨어를 포함한 핸들러 생성
handler := proxy.createCachingHandler()

proxy.handler = handler
return proxy, nil
}

func (cp *CachingProxy) createCachingHandler() http.Handler {
baseHandler := createGatewayHandler(cp.backend)

return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
// 캐시 키 생성
cacheKey := r.URL.Path + "?" + r.URL.RawQuery

// 캐시에서 확인
cp.mu.RLock()
if cached, exists := cp.cache[cacheKey]; exists {
cp.mu.RUnlock()
```

```
log.Printf("캐시 히트: %s", cacheKey)
w.Header().Set("X-Cache", "HIT")
w.Write(cached)
return
}
cp.mu.RUnlock()

// 캐시 미스 - 원본 핸들러 호출
log.Printf("캐시 미스: %s", cacheKey)
w.Header().Set("X-Cache", "MISS")

// ResponseWriter를 래핑하여 응답을 캐시에 저장
recorder := &responseRecorder{
ResponseWriter: w,
body: &bytes.Buffer{},
}

baseHandler.ServeHTTP(recorder, r)

// 성공적인 응답만 캐시에 저장
if  recorder.statusCode == 0 || recorder.statusCode == http.StatusOK {
cp.mu.Lock()
cp.cache[cacheKey] = recorder.body.Bytes()
```

```
cp.mu.Unlock()
log.Printf("응답 캐시됨: %s (%d bytes)", cacheKey, recorder.body.Len())
}
})
}

// responseRecorder는 응답을 기록하는 ResponseWriter입니다
type responseRecorder struct {
http.ResponseWriter
body *bytes.Buffer
statusCode  int
}

func (rr *responseRecorder) Write(data []byte) (int, error) {
rr.body.Write(data)
return  rr.ResponseWriter.Write(data)
}

func (rr *responseRecorder) WriteHeader(statusCode  int) {
rr.statusCode = statusCode
rr.ResponseWriter.WriteHeader(statusCode)
}
```

3. 모니터링 및 메트릭

```
// MetricsCollector는 프록시 메트릭을 수집합니다
type MetricsCollector struct {
requestCount prometheus.Counter
requestDuration prometheus.Histogram
cacheHitRate prometheus.Gauge
backendErrors prometheus.Counter
}

// NewMetricsCollector는 새로운 메트릭 수집기를 생성합니다
func  NewMetricsCollector() *MetricsCollector {
mc := &MetricsCollector{
requestCount: prometheus.NewCounter(prometheus.CounterOpts{
Name: "gateway_proxy_requests_total",
Help: "총 요청 수",
},
requestDuration: prometheus.NewHistogram(prometheus.HistogramOpts{
Name: "gateway_proxy_request_duration_seconds",
Help: "요청 처리 시간",
Buckets: prometheus.DefBuckets,
},
cacheHitRate: prometheus.NewGauge(prometheus.GaugeOpts{
```

```
Name: "gateway_proxy_cache_hit_rate",
Help: "캐시 히트율",
},
backendErrors: prometheus.NewCounter(prometheus.CounterOpts{
Name: "gateway_proxy_backend_errors_total",
Help: "백엔드 오류 수",
},
}

// Prometheus에 메트릭 등록
prometheus.MustRegister(mc.requestCount)
prometheus.MustRegister(mc.requestDuration)
prometheus.MustRegister(mc.cacheHitRate)
prometheus.MustRegister(mc.backendErrors)

return mc
}

// WrapHandler는 핸들러를 메트릭 수집 미들웨어로 래핑합니다
func (mc *MetricsCollector) WrapHandler(handler http.Handler) http.Handler {
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
start := time.Now()
```

```
// 요청 수 증가
mc.requestCount.Inc()


// 원본 핸들러 호출
handler.ServeHTTP(w, r)


// 처리 시간 기록
duration := time.Since(start).Seconds()
mc.requestDuration.Observe(duration)


log.Printf("요청 처리됨: %s %s (%.3fs)", r.Method, r.URL.Path, duration)
})
}
```

실행 방법

1. 빌드

```
# 프로젝트 디렉토리에서
go mod init gateway-proxy-example
go mod tidy
go build -o gateway-proxy
```

2. 블록 기반 프록시 실행

```
# 기본 포트(8040)에서 블록 기반 프록시 시작
./gateway-proxy -g https://trustless-gateway.link -type blocks

# 사용자 정의 포트에서 실행
./gateway-proxy -g https://trustless-gateway.link -type blocks -p 8080
```

3. CAR 기반 프록시 실행

```
# CAR 기반 프록시 시작 (IPIP-0402 지원 게이트웨이 필요)
./gateway-proxy -g https://trustless-gateway.link -type car -p 8041
```

4. 테스트

```
# 이미지 파일 테스트
curl -I http://localhost:8040/ipfs/bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqab

# 웹사이트 테스트
curl -H 'Host: en.wikipedia-on-ipfs.org' http://127.0.0.1:8040/wiki/ | head

# 메트릭 확인
curl http://127.0.0.1:8040/debug/metrics/prometheus
```

사용 사례

1. 서브도메인 게이트웨이

프록시는 서브도메인 게이트웨이로 작동하여 Origin 격리를 제공합니다:

```
http://bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi.ipfs.localhos
```

2. 경로 게이트웨이

단순한 파일 호스팅을 위한 경로 게이트웨이:

```
http://127.0.0.1:8040/ipfs/bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy5
```

3. DNSLink 게이트웨이

DNSLink 웹사이트 호스팅:

```
curl -H 'Host: en.wikipedia-on-ipfs.org' 'http://127.0.0.1:8040/wiki/'
```

장점과 특징

블록 기반 프록시

- **장점**: 세밀한 블록 단위 검증, 효율적인 부분 콘텐츠 로딩
- **단점**: 많은 HTTP 요청, 네트워크 오버헤드

CAR 기반 프록시

- **장점**: 적은 HTTP 요청, 효율적인 대용량 콘텐츠 전송
- **단점**: IPIP-0402 지원 게이트웨이 필요, 전체 CAR 파일 다운로드

공통 장점

1. **검증 가능성**: 모든 데이터의 암호학적 검증
2. **신뢰성**: 신뢰할 수 없는 원격 게이트웨이 사용 가능
3. **호환성**: 표준 웹 브라우저에서 직접 사용 가능
4. **확장성**: 여러 백엔드 게이트웨이 지원

모범 사례

1. 오류 처리

```
// 적절한 오류 응답 제공
if  err != nil {
http.Error(w, fmt.Sprintf("내부 서버 오류: %v", err), http.StatusInternalServerErr
return
}
```

2. 타임아웃 설정

```
// 적절한 타임아웃 설정
server := &http.Server{
ReadTimeout: 30 * time.Second,
```

```
    WriteTimeout: 30 * time.Second,

    }
```

3. 로깅

```
// 구조화된 로깅 사용

log.Printf("요청 처리: %s %s -> %d (%v)", r.Method, r.URL.Path, statusCode, durat
```

4. 보안

```
// CORS 헤더 적절히 설정

handler = gateway.NewHeaders(nil).ApplyCors().Wrap(handler)
```

문제 해결

일반적인 문제와 해결책

1. **원격 게이트웨이 연결 실패**

- 게이트웨이 URL 확인
- 네트워크 연결 상태 점검
- 방화벽 설정 확인

2. **CAR 형식 지원 오류**

- IPIP-0402 지원 게이트웨이 사용 확인
- 블록 기반 프록시로 대체 시도

3. **성능 문제**

- 캐싱 구현
- 여러 백엔드 게이트웨이 사용
- 적절한 타임아웃 설정

4. **메모리 사용량 증가**

- 캐시 크기 제한 설정
- 주기적인 캐시 정리
- 스트리밍 응답 사용

이 예제를 통해 신뢰할 수 있는 IPFS Gateway 프록시를 구현할 수 있으며, 실제 프로덕션 환경에서는 추가적인 보안, 모니터링, 로드 밸런싱 기능을 고려해야 합니다.